WAB Exposé

Provadis School of International Management and Technology

Performance of (Different) Python Runtimes for a Bioinformatics Algorithm

Testing the Smith-Waterman algorithm across CPython, PyPy, and Python 3.14's native JIT

Rubin Chempananickal James
rubin.chempananickal-james@stud-provadis-hochschule.de
Matriculation Number: D876

Department: Information Technology
Module: Fortgeschrittene Programmierung
Reviewer: Prof. Dr. Jörg Daubert

25.03.2026

Contents

Exposé 1
1. Introduction 1
2. Research Question 2
3. Hypothesis 2
4. Objectives 2
5. Methodology 2
6. Planned Structure 3
References i

Exposé

1. Introduction

Python [1] has emerged to be the most widely used programming language among many fields, including bioinformatics[2]. It has many strengths, such as a very beginner friedly syntax, an exceptionally large ecosystem of packages, dynamic typing, and a feature rich standard library. One area where it is clearly lacking, however, is performance. This is due to the fact that the default implementation of Python, CPython, is interpreted and dynamically typed, which leads to significant overhead at runtime.

> The Python interpreter does a lot of work to try to abstract away the underlying computing elements that are being used. At no point does a programmer need to worry about allocating memory for arrays, how to arrange that memory, or in what sequence it is being sent to the CPU. This is a benefit of Python, since it lets you focus on the algorithms that are being implemented. However, it comes at a huge performance cost.
>
> — [3, p. 11]

Many solutions have been proposed to address this issue while still remaining within the Python ecosystem. Chief among them are:

- Transpilation into C using Cython [4]
- Tracing Just-in-Time (JIT) compilation, as implemented in an alternative Python interpreter to CPython, called PyPy [5]
- Copy-and-patch JIT compilation, as proposed in [6], and with the latest stable implementation in Python 3.14 [7].

The native JIT approach is by far the newest, and it's still under active development and marked as experimental.

The Smith-Waterman (SW) algorithm [8] is a very common algorithm in bioinformatics. It is a local alignment algorithm that computes the highest-scoring matching subsequence(s) between two DNA/RNA/protein sequences. It is a very computationally intensive algorithm, with a time complexity of O(m*n) for two sequences of length m and n, so a naive implementation in Python is expected to perform very poorly. The algorithm nevertheless has two nested "hot loops", which the performance-minded runtimes could potentially optimize. That, and the

fact that it is still the best non-heuristic local alignment algorithm, and that it is relatively simple to implement (fewer than 100 lines of code), were the reasons why it was chosen as the algorithm for this performance comparison.

2. Research Question

How does the SW algorithm perform across the different Python runtimes, namely the default CPython interpreter, the newer CPython JIT, PyPy, and Cython?

3. Hypothesis

The hypothesis is that either Cython or PyPy will outperform the rest by a wide margin in terms of execution time. PyPy is expected to show a warm-up effect: early runs should be slower, while later runs should become faster once the tracing JIT compiler has optimized frequently executed code paths. Afterward, performance is expected to stabilize. Cython's execution time should be very fast from the first run, since it is transpiled into C and compiled ahead of time.

The native JIT implementation is still in its infancy, therefore it may not perform better than PyPy and Cython, but it is still expected to outperform the default CPython interpreter.

4. Objectives

The main objectives of this paper are as follows:

- To compare the performance of the SW algorithm across CPython, CPython with native JIT, PyPy, and Cython.
- To evaluate how different runtime and compilation strategies affect execution time for this computationally intensive bioinformatics algorithm.
- To identify the strengths and limitations of each runtime for this specific use case.

5. Methodology

The benchmarking methodology is designed to make the comparison between runtimes as fair and reproducible as possible. The same SW implementation will be used across the default CPython interpreter, CPython with native JIT enabled, and PyPy.

For Cython, the same algorithm will be transferred into a corresponding `.pyx` implementation with type annotations where necessary, and compiled as a native extension module with Cython and `setuptools`. Since this test will be run on

Windows 11, this build step relies on the Microsoft C++ build tools. The compiled module is then loaded from CPython and executed through the same benchmark interface as the pure Python implementations, so that the main difference under comparison is the compilation strategy rather than a completely different program structure.

A list of DNA sequence pairs benchmark cases will be generated synthetically, and will include a few specific scenarios likely to be encountered in real-world bioinformatics applications, in addition to random string pairs. The benchmark is likely to target sequences of lengths from 100 to 1000 in increments of 200, since much larger sequences would lead to prohibitively long execution times as the time complexity is quadratic. To ensure a fair comparison, the exact same generated cases will be reused across all runtimes.

Each case will then be executed on four runtimes sequentially: CPython, CPython with native JIT, PyPy, and Cython. For every case-runtime combination, two warm-up runs will be executed first and then ten timed runs per sequence pair will be measured. The median of these timed runs will be used as the representative execution time for that case in order to reduce the influence of outliers. In addition, the alignment scores produced by each runtime will be checked against the CPython baseline to ensure that incorrectness is not mistaken for performance differences.

The collected raw runtime measurements will then be aggregated by scenario and problem size, and analyzed with statistical methods and visualizations.

6. Planned Structure

The paper will likely be structured as follows:

- Introduction
- Background and Related Work
- Methodology
- Results
- Discussion
- Conclusion and Future Work
- References
- AI Declaration
- Declaration of Authorship

References

[1] G. Van Rossum and F. L. Drake, *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009.

[2] D. Mariano, M. Ferreira, B. L. Sousa, L. H. Santos, and R. C. de Melo-Minardi, "A Brief History of Bioinformatics Told by Data Visualization," in *Advances in Bioinformatics and Computational Biology*, J. C. Setubal and W. M. Silva, Eds., Cham: Springer International Publishing, 2020, pp. 235–246. doi: 10.1007/978-3-030-65775-8_22.

[3] M. Gorelick and I. Ozsvald, *High performance python*. O'Reilly Media Inc., 2025.

[4] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith, "Cython: The Best of Both Worlds," *Computing in Science & Engineering*, vol. 13, no. 2, pp. 31–39, 2011, doi: 10.1109/MCSE.2010.118.

[5] The PyPy Team, "PyPy Performance." Accessed: Mar. 25, 2026. [Online]. Available: https://www.pypy.org/performance.html

[6] B. Bucher and S. Ostrowski, "PEP 744 – JIT Compilation." Accessed: Mar. 10, 2026. [Online]. Available: https://peps.python.org/pep-0744/

[7] Python Software Foundation, "What's New In Python 3.14." Accessed: Mar. 10, 2026. [Online]. Available: https://docs.python.org/3/whatsnew/3.14.html

[8] T. F. Smith and M. S. Waterman, "Identification of common molecular subsequences," *Journal of Molecular Biology*, vol. 147, no. 1, pp. 195–197, 1981, doi: 10.1016/0022-2836(81)90087-5.